

AVANCE DE PROYECTO FINAL

Estructura de datos

Docente: Alberto Lemus Mera



ÍNDICE

Objetivo.....	1
Instrucciones.....	1
Código.....	2
Node.java.....	2
Lista.java.....	2
Cola.java.....	3
Pila.java.....	5
Tarea.java.....	7
App.java.....	8
Código corriendo.....	9

Objetivo

Implementar y manipular diferentes estructuras de datos en Java, así como entender las aplicaciones prácticas de pilas, colas y listas en el manejo de información para desarrollar habilidades de programación y resolución de problemas en un contexto empresarial simulado.

Instrucciones

1. **Revisar conceptos básicos:** antes de empezar a programar, revisa la información sobre pilas, colas, listas y una introducción a las estructuras de datos para refrescar tus conocimientos.
2. **Entorno de desarrollo:** asegúrate de tener Java y Apache Netbeans instalados en tu computadora. Además, configura tu entorno de desarrollo para iniciar el proyecto.
3. **Diseño del programa:**
 1. **Estructura del programa:** divide tu programa en tres partes principales, cada una correspondiente a una estructura de datos (pila, cola, lista).
 2. **Pilas:** utiliza pilas para manejar tareas urgentes que deben realizarse lo antes posible. Implementa operaciones básicas como *push*, *pop* y *peek*.
 3. **Colas:** emplea colas para administrar tareas en una secuencia FIFO (primero en entrar, primero en salir), ideal para tareas programadas regularmente. Implementa operaciones básicas como **enqueue**, **dequeue** y **front**.
 4. **listas:** usa listas para tareas que pueden requerir acceso aleatorio, como la consulta de tareas pendientes por departamento. Implementa operaciones básicas como **insert**, **delete** y **find**.
4. **Funcionalidad adicional:**
 1. Implementa una interfaz de usuario sencilla en la consola para interactuar con el sistema.
 2. Permite al usuario agregar, ver y eliminar tareas en cada estructura de datos.
 3. Incluye una opción para ver todas las tareas pendientes, ordenadas por urgencia y departamento.
5. **Pruebas:** realiza varias pruebas para asegurarte de que todas las operaciones funcionan correctamente en cada estructura de datos.

Código

Node.java

```
public class Node<T> {
    T data;
    Node<T> next;
    Node<T> prev;

    public Node(T data) {
        this.data = data;
        this.next = null;
        this.prev = null;
    }

    public String toString() {
        return data != null ? data.toString() : "null";
    }
}
```

Lista.java

```
public class Lista<T> {
    public Node<T> head;

    public Lista() {
        this.head = null;
    }

    public void insert(T data) {
        Node<T> newNode = new Node<>(data);
        if (head == null) {
            head = newNode;
        } else {
            Node<T> current = head;
            while (current.next != null) {
                current = current.next;
            }
            current.next = newNode;
            newNode.prev = current;
        }
    }

    public T find(T target) {
```

```
Node<T> current = head;
while (current != null) {
    if (current.data.equals(target)) {
        return current.data;
    }
    current = current.next;
}
return null;
}

public boolean delete(T target) {
    Node<T> current = head;
    while (current != null) {
        if (current.data.equals(target)) {
            if (current.prev != null) {
                current.prev.next = current.next;
            } else {
                head = current.next;
            }
            if (current.next != null) {
                current.next.prev = current.prev;
            }
            return true;
        }
        current = current.next;
    }
    return false;
}
}
```

Cola.java

```
public class Cola<T> {
    public Node<T> front;
    public Node<T> back;
    public int size;

    public Cola() {
```

```
        this.front = null;
        this.back = null;
        this.size = 0;
    }

    public boolean isEmpty() {
        return front == null;
    }

    public void enqueue(T data) {
        Node<T> nuevoNode = new Node<>(data);
        if (isEmpty()) {
            front = nuevoNode;
            back = nuevoNode;
        } else {
            back.next = nuevoNode;
            back = nuevoNode;
        }
        size++;
    }

    public T dequeue() {
        if (isEmpty()) {
            System.out.println("Queue underflow: Nada de que mostrar");
            return null;
        }
        T dataRemovido = front.data;
        front = front.next;
        if (front == null) {
            back = null;
        }
        size--;
        return dataRemovido;
    }

    public T front() {
        if (isEmpty()) {
            System.out.println("Queue underflow: Nada de que mostrar");
            return null;
        }
    }
```

```
        return front.data;
    }

    public int getSize() {
        return size;
    }

    public void mostrarCola() {
        if (isEmpty()) {
            System.out.println("Nada que mostrar");
        }
        Node<T> nodoActual = front;
        System.out.print("Primer elemento -> ");
        while (nodoActual != null) {
            System.out.println(nodoActual.data + " -> ");
            nodoActual = nodoActual.next;
        }
        System.out.println("Ultimo elemento");
    }
}
```

Pila.java

```
public class Pila<T> {
    public Node<T> top;
    public int size;

    public Pila() {
        this.top = null;
        this.size = 0;
    }

    public boolean isEmpty() {
        return top == null;
    }

    public void push (T data) {
        Node<T> nuevoNodo = new Node<>(data);
```

```
nuevoNodo.next = top;
top = nuevoNodo;
size++;
}

public T pop() {
    if (isEmpty()) {
        System.out.println("Pila vacia");
        return null;
    }
    T datoRemovido = top.data;
    top = top.next;
    size--;
    return datoRemovido;
}

public T peek() {
    if (isEmpty()) {
        System.out.println("Pila vacia");
    }
    return top.data;
}

public int getSize() {
    return size;
}

public void mostrarPila() {
    if (isEmpty()) {
        System.out.println("Pila vacia");
    }
    Node<T> nodoActual = top;
    System.out.print("Cima de la pila -> ");
    while (nodoActual != null) {
        System.out.println(nodoActual.data + " -> ");
        nodoActual = nodoActual.next;
    }
    System.out.println("Fondo de la pilas");
}
}
```

Tarea.java

```
public class Tarea {
    public String descripcion;
    public String departamento;
    public int urgencia; // 1 = Alta (Urgente), 2 = Media (Regular), 3 =
    Baja (Lista)

    public Tarea(String descripcion, String departamento, int urgencia) {
        this.descripcion = descripcion;
        this.departamento = departamento;
        this.urgencia = urgencia;
    }

    public int getUrgencia() {
        return urgencia;
    }

    public String getDepartamento() {
        return departamento;
    }

    @Override
    public String toString() {
        String nivel = (urgencia == 1) ? "URGENTE" : (urgencia == 2) ?
"REGULAR" : "NORMAL";
        return "[" + nivel + "] Depto: " + departamento + " | Tarea: " +
descripcion;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null || getClass() != obj.getClass()) return false;
        Tarea otraTarea = (Tarea) obj;
        return this.descripcion.equalsIgnoreCase(otraTarea.descripcion);
    }
}
```


App.java

```
public class Tarea {
    public String descripcion;
    public String departamento;
    public int urgencia; // 1 = Alta (Urgente), 2 = Media (Regular), 3 =
Baja (Lista)

    public Tarea(String descripcion, String departamento, int urgencia) {
        this.descripcion = descripcion;
        this.departamento = departamento;
        this.urgencia = urgencia;
    }

    public int getUrgencia() {
        return urgencia;
    }

    public String getDepartamento() {
        return departamento;
    }

    @Override
    public String toString() {
        String nivel = (urgencia == 1) ? "URGENTE" : (urgencia == 2) ?
"REGULAR" : "NORMAL";
        return "[" + nivel + "] Depto: " + departamento + " | Tarea: " +
descripcion;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null || getClass() != obj.getClass()) return false;
        Tarea otraTarea = (Tarea) obj;
        return this.descripcion.equalsIgnoreCase(otraTarea.descripcion);
    }
}
```

Código corriendo

```
java App.java

----- MENU HOSPITAL GENERAL -----
1. Tareas Urgentes (Pila)
2. Tareas Programadas (Cola)
3. Tareas por Departamento (Lista)
4. Ver todas las tareas pendientes (Ordenadas)
5. Salir
Introduce una opcion:
1

--- TAREAS URGENTES (Pila) ---
1. Agregar tarea (Push)
2. Atender tarea más reciente (Pop)
3. Ver próxima urgencia (Peek)
4. Volver
1
Descripción de la urgencia:
Cirugia de corazon
Departamento:
Cardiologia

--- TAREAS URGENTES (Pila) ---
1. Agregar tarea (Push)
2. Atender tarea más reciente (Pop)
3. Ver próxima urgencia (Peek)
4. Volver
3
Siguiendo en pila: [URGENTE] Depto: Cardiologia | Tarea: Cirugia de corazon

--- TAREAS URGENTES (Pila) ---
1. Agregar tarea (Push)
2. Atender tarea más reciente (Pop)
3. Ver próxima urgencia (Peek)
4. Volver
4

----- MENU HOSPITAL GENERAL -----
1. Tareas Urgentes (Pila)
2. Tareas Programadas (Cola)
3. Tareas por Departamento (Lista)
4. Ver todas las tareas pendientes (Ordenadas)
5. Salir
Introduce una opcion:
█
```

```
java App.java

Introduce una opcion:
3

--- TAREAS POR DEPARTAMENTO (Lista) ---
1. Agregar tarea (Insert)
2. Buscar tarea por descripción (Find)
3. Eliminar tarea (Delete)
4. Mostrar todas las tareas de esta lista
5. Volver
1
Descripción de la tarea:
Reunion de informacion
Departamento (Ej. Cardiología, Urgencias):
Cardiologia 3
Tarea registrada en el sistema.

--- TAREAS POR DEPARTAMENTO (Lista) ---
1. Agregar tarea (Insert)
2. Buscar tarea por descripción (Find)
3. Eliminar tarea (Delete)
4. Mostrar todas las tareas de esta lista
5. Volver
4

--- Lista de Tareas por Departamento ---
[NORMAL] Depto: Cardiologia 3 | Tarea: Reunion de informacion

--- TAREAS POR DEPARTAMENTO (Lista) ---
1. Agregar tarea (Insert)
2. Buscar tarea por descripción (Find)
3. Eliminar tarea (Delete)
4. Mostrar todas las tareas de esta lista
5. Volver
2
Ingresa la descripción exacta de la tarea a buscar:
Reunion de informacion
Tarea encontrada → [NORMAL] Depto: Cardiologia 3 | Tarea: Reunion de informacion

--- TAREAS POR DEPARTAMENTO (Lista) ---
1. Agregar tarea (Insert)
2. Buscar tarea por descripción (Find)
3. Eliminar tarea (Delete)
4. Mostrar todas las tareas de esta lista
5. Volver
```

```
java App.java
1. Tareas Urgentes (Pila)
2. Tareas Programadas (Cola)
3. Tareas por Departamento (Lista)
4. Ver todas las tareas pendientes (Ordenadas)
5. Salir
Introduce una opcion:
4

--- TODAS LAS TAREAS PENDIENTES ---
[URGENTE] Depto: Cardiologia | Tarea: Cirugia de corazon
[REGULAR] Depto: General | Tarea: Revision General
[NORMAL] Depto: Cardiologia 3 | Tarea: Reunion de informacion

----- MENU HOSPITAL GENERAL -----
1. Tareas Urgentes (Pila)
2. Tareas Programadas (Cola)
3. Tareas por Departamento (Lista)
4. Ver todas las tareas pendientes (Ordenadas)
5. Salir
Introduce una opcion:
1

--- TAREAS URGENTES (Pila) ---
1. Agregar tarea (Push)
2. Atender tarea más reciente (Pop)
3. Ver próxima urgencia (Peek)
4. Volver
2
Atendiendo: [URGENTE] Depto: Cardiologia | Tarea: Cirugia de corazon

--- TAREAS URGENTES (Pila) ---
1. Agregar tarea (Push)
2. Atender tarea más reciente (Pop)
3. Ver próxima urgencia (Peek)
4. Volver
4

----- MENU HOSPITAL GENERAL -----
1. Tareas Urgentes (Pila)
2. Tareas Programadas (Cola)
3. Tareas por Departamento (Lista)
4. Ver todas las tareas pendientes (Ordenadas)
5. Salir
Introduce una opcion:
```